Hazard Analysis
Keystone

Team 13

Ayush Patel
Ahmed Rashrash
Andrew Habib
Waqar Ul-Hassan

Table 1: Revision History

Date	Developer(s)	Change
Oct 18th	Ayush Patel	Introduction, Scope, System Boundary and Assumptions
Oct 19th	Ahmed Rashrash	FMEA Table, Safety and Security Requirements
Oct 19th	Ayush Patel	Roadmap for Safety Req Implementation
Oct 29th	Ayush Patel	Descriptive identifiers for hazards and SSRs, links into the SRS

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **1**
3.1 System Boundary . . . 1
3.2 Major Components . . . 2

4 Critical Assumptions **3**

5 Failure Mode and Effect Analysis **4**

6 Safety and Security Requirements **9**

7 Roadmap **13**
7.1 Phase 1: Core Workflow (implemented in the course) . . . 13
7.2 Phase 2: Communication and Visibility (implemented in the course) . . . 13
7.3 Phase 3: Extended Features (implemented if time allows) . . . 14
7.4 Future Work (not implemented, or faked for the demo) . . . 14

8 Appendix — Generative AI Use Disclosure **15**

9 Appendix — Reflection **16**

List of Figures

1 System and boundary context diagram for Keystone . . . 2

List of Tables

1 Revision History . . . i
2 Failure Mode and Effect Analysis . . . 4

1 Introduction

Keystone is a capstone project discovery and management platform intended to support the lifecycle of a capstone project, from project proposal and discovery through team formation, project approval, and the management of project information. The platform is intended to serve students, project supervisors, course instructors, and teaching assistants (TAs).

Although Keystone is not intended to control physical equipment or perform safety-critical operations, failures can still cause harm. Examples include unauthorized disclosure of confidential project and student information, invalid team assignments, lost or inconsistent project updates, and misleading approval or notification status. These failures could delay academic activities, create unnecessary work, and damage trust.

We define a hazard as an unsafe system condition that, in combination with particular environmental circumstances, can contribute to an undesirable outcome. Hazard analysis examines how system failures and interactions can produce such conditions and identifies measures to eliminate or control them (Ippolito and Wallace, 1995; Leveson, 2011).

This document identifies potential hazards associated with Keystone, analyzes the ways in which system components may fail, and proposes mitigations that can be translated into verifiable safety and security requirements.

Hazards and safety and security requirements are identified by descriptive names. Hazard names (prefixed `H_`) and safety and security requirement names (prefixed `SSR_`) link to their definitions in this document. Names of requirements in the SRS (prefixed `FR_`, `NFR_`, and so on) link to the corresponding requirement in the published SRS.

2 Scope and Purpose of Hazard Analysis

The scope includes functional failures, integration failures, security and privacy weaknesses, usability and accessibility issues, and foreseeable user errors. It also considers failures at the boundaries between Keystone and external services.

This hazard analysis covers the following categories of loss:

- **Disruption of academic activities:** Failure to discover projects, form eligible teams, obtain approvals, or maintain accurate project records may delay project selection and interfere with course milestones.
- **Unauthorized disclosure or misuse of information:** Confidential project proposals, intellectual property concerns, contact information, student profiles, and team records may be exposed to people who are not authorized to access them.
- **Incorrect academic or administrative decisions:** Inaccurate project descriptions, invalid team assignments, incorrect course rules, or misleading approval statuses may cause students, supervisors, or instructors to make decisions based on incorrect information.
- **Loss or corruption of project information:** Failed updates, integration errors, or inconsistent records may cause project details, supervisor information, repository links, or selected documentation requirements to become unavailable or outdated.
- **Loss of trust and adoption:** Unreliable notifications, unclear workflows, inaccessible interfaces, or inappropriate disclosure of information may frustrate users and reduce confidence in the platform.

3 System Boundaries and Components

3.1 System Boundary

Keystone is the system under analysis. Its boundary includes the application logic, persistent data, user-facing functionality, and integrations implemented or configured specifically for the platform. See Figure 1

External services are not assumed to be under Keystone's direct control. Nevertheless, their interfaces and the consequences of their failure are included in the analysis because Keystone depends on them to provide its intended functionality.

The following external entities and services are relevant:

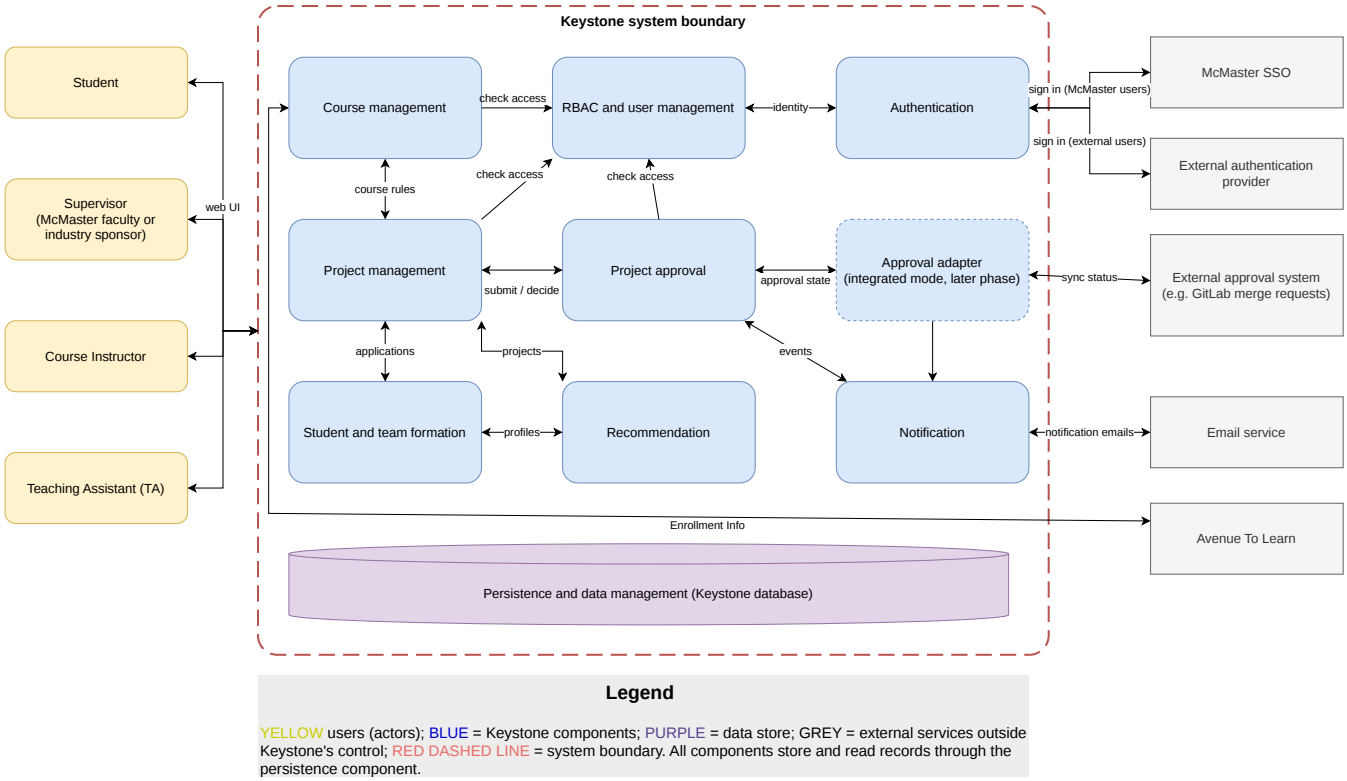


Figure 1: System and boundary context diagram for Keystone

- **McMaster single sign-on (SSO):** Provides authentication for users accessing Keystone through their McMaster credentials.
- **External authentication provider:** Supports authentication for users who do not have a McMaster account.
- **Email service:** Delivers notifications concerning project submissions, approvals, supervisor confirmations, and other events.
- **External approval system:** A course-specific system outside Keystone where a course approves team projects, for example the GitLab merge request process used by the Software Engineering course. It is not under Keystone's control. In *external* approval mode, the instructor or a TA records the approval outcome in Keystone by hand. In *integrated* mode, an approval adapter (a Keystone component) reads the approval state through the external approval system's interface and updates Keystone's records.
- **Avenue to Learn:** McMaster's learning management system. Keystone reads a course's enrollment list from it, for courses that have an Avenue site. It is not under Keystone's control and may be unavailable, out of date, or incomplete.

3.2 Major Components

The following components define the principal areas in which failures will be investigated. They are meant to be functional boundaries for hazard identification, rather than a complete software architecture.

1. **Authentication component:** Manages user sign-in through McMaster SSO and an external authentication provider. It is responsible for establishing the identity associated with a user session. Authentication failures may allow unauthorized access or prevent legitimate users from accessing the platform.
2. **Role-Based Access Control (RBAC) and user management component:** Maintains user accounts, roles, and permissions. It determines which resources users may view or modify. Relevant roles include students, supervisors, TAs, and course instructors. Incorrect permissions may expose restricted information or allow unauthorized changes.

3. **Course management component:** Maintains course configurations, course membership, instructor and TA assignments, course-specific rules and configurations, and relevant deadlines or course information. It must accommodate differences between Software Engineering (SE), Computer Science (CS), and Mechatronics (TRON) courses.
4. **Project management component:** Manages project proposals, descriptions, supervisors, contact details, project categories, confidentiality information, intellectual property (IP) concerns, repository links, and project status throughout the project lifecycle.
5. **Project approval component:** Supports project proposal review, team project approval, and the recording of approval decisions. It must distinguish between submission, review, approval, and other workflow states so that users are not misled about whether a project or team has been approved.
6. **Student and team formation component:** Manages student profiles, team membership, project interests, and any supported team or project recommendations. It must respect course eligibility rules and team-size constraints.
7. **Notification component:** Generates and sends notifications associated with relevant workflow events via the email service. Failures may result in missing, delayed, duplicated, or incorrectly addressed messages.
8. **Persistence and data management component:** Stores and retrieves the platform's persistent records. It maintains the consistency of related information, including project records, team membership, course configuration, approval states, and user permissions.
9. **Approval adapter (integrated mode):** Reads approval state from an external approval system and updates Keystone's records. Failures may leave Keystone showing a state that differs from the external system.
10. **Recommendation component:** Ranks or filters projects and potential team matches based on available information and user preferences. Incorrect recommendations may hide relevant opportunities or mislead users about their suitability for a project.

4 Critical Assumptions

1. **Instructor authority:** Course instructors remain responsible for final project approval, the interpretation of course rules, and exceptional team-formation decisions. Keystone supports these decisions but does not replace the instructor's judgment regarding project feasibility, scope, or academic suitability.
2. **Authentication provider availability:** McMaster SSO and any external authentication provider are external dependencies and may become unavailable. Keystone cannot guarantee their continuous operation but must handle authentication failures safely.
3. **Accuracy of user-supplied information:** Students and supervisors may submit incomplete, incorrect, misleading, or outdated information. The platform can validate information according to defined rules, but it cannot guarantee the truth of every project description, supervisor commitment, or user-provided statement.
4. **Feature configuration:** Optional features may be disabled by course instructors to support different capstone courses or future deployments. Disabling a feature must not leave dependent workflows in an invalid state or expose information that would otherwise be restricted.
5. **Enrollment data availability and accuracy:** Avenue to Learn is an external dependency, and not every course has an Avenue site. Its enrollment data may be unavailable, out of date, or incomplete. Keystone cannot guarantee that an imported list is correct, so course staff remain responsible for confirming imports and for adding students who are missing.

5 Failure Mode and Effect Analysis

Table 2: Failure Mode and Effect Analysis

Component	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
Authentication	SSO or external provider unavailable	Users cannot access platform, leads to missed deadlines	External service outage, network failure	Display error, log failure, raise warning to IT department and/or instructors	SSR_ExternalErrorLogging	H_AuthProviderOutage
	User session is stolen	Attacker can act as the user, read sensitive data, perform unapproved actions	Compromised device, unencrypted user traffic, compromised external service	Re-verify the user before every action on the closed list in SSR_ReverifyCriticalActions	SSR_ReverifyCriticalActions	H_SessionTheft
RBAC and user management	User is assigned the wrong role	Student able to act as a course instructor, supervisor as an IT technician, etc.	System inferring roles from an unreliable source	Default to zero privileges if automatic assignment fails, provide instructor with ability to modify roles, and record all actions in an audit log	SSR_AuditLog, SSR_ZeroDefaultPrivileges	H_WrongRole
	A person with no connection to the course signs in through the external provider and is treated as a supervisor	Student information and student-proposed projects are disclosed to outsiders, and the platform can be scraped	The external provider accepts any identity, and the role is assigned at sign-in	Give every external sign-in the unverified external supervisor role, limited to course reference material, the proposal template, and their own proposals, and verify it only when an instructor approves a proposal that the account submitted	SSR_ExternalSupervisorVerification, SSR_ZeroDefaultPrivileges	H_UnvettedExternalAccess
	An external supervisor reads another sponsor's confidential project	Loss of confidentiality of industry project ideas, loss of sponsor trust, IP disputes	Visibility rules written for "course members" applied to all supervisors, or other sponsors' projects exposed through search, counts, status, or email	Scope every read path for an external supervisor to the projects they proposed or are named on, and answer other requests as not found	SSR_ExternalSupervisorScope, SSR_EnforceVisibility	H_CrossSponsorExposure

Continued on next page

Component	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
Course management	Disabling a feature leaves dependent workflows in an invalid state	Other components fail unnecessarily and system rejects normal actions	Component state does not account for all possible configurations	Reject disable request, advise user on actions needed before feature can be disabled	SSR_RejectDisable, SSR_NotifyPendingActions	H_DisableBreaksWorkflow
	Team formation rules are misconfigured, or changed after teams have formed	Valid teams are blocked, or invalid teams are accepted; teams that were valid under the old rules become non-compliant without notice	Wrong team size limits or program combinations entered, rule edits applied without checking existing teams	Validate the rule configuration on save, restrict edits to instructors, preview the teams a change would make non-compliant, and flag those teams for instructor review instead of invalidating them	SSR_RuleChangeImpact, SSR_AuditLog	H_RuleMisconfiguration
Project management	Project displays outdated information	Teams operate based on false information, contacting individuals which may no longer be involved with the course	Instructor, supervisor, or even students forget to update their information	Display a timestamp for when any displayed records were last updated	SSR_RecordTimestamps	H_OutdatedProject
	Confidential project or IP exposed	Potential monetary loss, loss of trust, exposed vulnerabilities	Project visibility rules not being enforced across a project	Check every way of reading a record (catalog, search, project detail, threads, emails, exports, recommendations) on the server against the visibility matrix in the SRS	SSR_EnforceVisibility	H_ConfidentialExposure
	Project description misrepresents project	Team cannot move forward with project, time wasted	Project proposal prompt not asking for critical information	Require every attribute that the SRS data dictionary marks as Required and prompt for the optional ones	SSR_ProposalPromptCoverage	H_MisleadingDescription

Continued on next page

Component	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
	A supervisor accepts a team and takes the last slot, but the project is not shown as Full (or a released slot does not reopen it)	Other teams apply to or wait on a project that cannot take them	Failure between reserving the slot and updating the project status or notifying applicants	Reserve the slot, update the project status, and queue notifications to teams with open applications in a single transaction	SSR_AtomicActions	H_StaleProjectStatus
Project approval	A team or supervisor sees "Approved" when the team-project status is Agreed, Submitted, or Under review, or a post-approval change replaces the approved record before the instructor accepts it	Team acts on an approval it does not have, or the official record is wrong	Displayed status derived separately from the stored status, change requests written directly to the approved record	Display only the stored team-project status using the lifecycle labels, hold changes to an approved record as pending, and apply them only when the instructor accepts	SSR_ApprovalStatusIntegrity	H_MisleadingApprovalStatus
Approval adapter (integrated mode)	Synchronization with the external approval system is stale or fails	Keystone shows a status that differs from the external system, a team misses a deadline or is treated as approved or unapproved incorrectly	External system outage or interface change, adapter error, delayed polling	Show the time of the last successful sync on externally approved records, flag failed syncs, and let the instructor or TA record the outcome manually (logged)	SSR_AdapterSyncStatus	H_AdapterSyncFailure
Student and team formation	Two students attempt to join the last slot of a group simultaneously, neither receives a fail message	Both students believe they have joined the group, team size limit may be exceeded due to the data race	Database allows both operations to run concurrently without a locking mechanism	Ensure that data entries that are part of a single operation are not modified externally for the total duration of the operation	SSR_NoStaleReads	H_TeamSizeRace

Continued on next page

Component	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
Student and team formation	Decline or rejection is communicated bluntly, or students feel publicly ranked	Stress, feeling unfairly treated, students stop using the platform	Free-text decline messages, visible statuses with no guidance	Use respectful message templates with an optional reason, and show no public ranking of students or teams	SSR_RespectfulDeclines	H_BluntRejection
Notification	Important notification is not delivered	User misses important deadline, fails to respond to time sensitive matter	Notification component fails in the process of sending a notification, queued notification is lost	Ensure that queued notifications remain available until delivery is verified to have completed	SSR_DurableNotifications	H_LostNotification
	Notification is sent to the wrong recipient	Confidential proposal, IP, or student information is disclosed to someone who should not see it	Recipients resolved from stale roles or associations, confidential fields placed in the email body	Resolve recipients at send time from current role and association, and include only an event summary and a link in the email body	SSR_NotificationRecipientScope	H_MisaddressedNotification
	Same notification delivered more than once	Users are confused or ignore later notifications, trust in the platform drops	Retries after an unconfirmed delivery	Give each notification a unique identifier so a retry never results in a second send	SSR_NotificationDeduplication	H_DuplicateNotification
Persistence and data management	Data is lost or corrupted	User data is lost, system fails to operate as expected, audit log lost	Hardware degradation, accidental deletion, migration failures	Regular, automated backups which are verified to be able to restore platform to a previous, working state	SSR_AutomatedBackups	H_DataLoss
	Personal data kept longer than needed	Privacy breach, loss of trust, legal exposure	No retention policy, deletion not implemented	Define a retention period and delete personal data on request and at the end of the course cycle	SSR_PersonalDataDeletion	H_ExcessRetention

Continued on next page

Component	Failure Mode	Effects of Failure	Causes of Failure	Recommended Action	SR	Ref
Recommendation	Irrelevant projects ranked highly	Users miss projects that interest them, due to the assumption that lower projects are not worth looking into	Recommendation algorithm performs badly, inputs are outdated	Provide multiple ways to sort project list	SSR_MultipleBrowseViews	H_IrrelevantRanking
Enrollment import	Avenue unavailable or the import fails	Course cannot be populated with its students, team validation cannot run, team formation and approval are delayed	Avenue outage, change to Avenue access, or the course has no Avenue site	Leave existing enrollment unchanged, tell the user, and offer file upload	SSR_EnrollmentFallback	H_ImportUnavailable
	Import produces a wrong list (missing, duplicate, or mismatched students)	Students wrongly blocked from or admitted to the course, invalid team validation results	Stale or partial data from Avenue, malformed identifiers	Check student identifiers, detect duplicates, and show an import preview for the user to review and confirm before applying changes.	SSR_EnrollmentPreview, SSR_EnrollmentValidation	H_ImportWrongList
	Student missing from the import is removed from a team automatically	Team becomes undersized or invalid in the middle of approval, student loses their place without notice	Import treated as the single source of truth and applied without review	Flag absent students for review and never remove them automatically	SSR_NoAutoRemoval	H_AutoRemovalFromTeam
	Student who joins late cannot be added	Student has no team and no way to use the platform, and team formation is delayed	The import is the only way to add students	Let course staff add students from an additional file import	SSR_EnrollmentFallback	H_LateStudent

6 Safety and Security Requirements

SSR_ExternalErrorLogging The system shall log all errors caused by external providers and notify university IT professionals and/or course instructors when they are detected.
Rationale: Outages caused by external providers prevent users from using the platform and require immediate attention to restore in a timely manner.
Fit Criterion: An authorized administrator can read a log that contains external provider errors, and an instructor or IT contact is notified when one occurs.
Traces To: FR_LogExternalErrors

SSR_AuditLog The system shall record all user actions with the time, a user identifier, and their privilege level at the time of the action, in an append-only audit log.
Rationale: Accounts may get compromised and any actions taken may need to be reviewed and reversed.
Fit Criterion: One action of each type (approval decision, status change, role change) produces an entry with time, user identifier, and privilege level, and no interface or API path edits or deletes entries.
Traces To: NFR_AuditLog, NFR_AuditTamper, FR_AuthorizationRBAC, FR_UpdateRoles

SSR_RejectDisable The system shall reject all component disable requests if there are pending actions within the component.
Rationale: Disabling features may potentially leave the system in a partially invalid state. For example, disabling the project approvals component strands pending project approval requests.
Fit Criterion: Attempting to disable the project approval component while at least one approval record is pending is rejected, and the pending records are listed.
Traces To: FR_RejectComponentDisable

SSR_RecordTimestamps The system shall display a timestamp for any displayed records when they were last updated.
Rationale: Providing users with a timestamp can allow them to judge whether the information they are reading is up-to-date and can contact instructors or supervisors when that is not the case.
Fit Criterion: Every project and profile page displays a last-updated timestamp that changes when the record is modified.
Traces To: FR_DisplayTimestamps

SSR_DurableNotifications All pending notifications shall be stored durably until delivery is verified.
Rationale: Lost notifications mean that users are not alerted of potentially time-sensitive matters and this can result in missed deadlines and loss of time.
Fit Criterion: If the system is stopped before a queued notification is sent, the notification is sent after restart.
Traces To: FR_DurableNotifications

SSR_AutomatedBackups The platform shall provide a mechanism for automatically creating regular backups which can restore the platform to a previous state.
Rationale: Data loss incidents are not uncommon and can be caused by faulty hardware, accidental changes, and many others. Storing a set of backups ensures that any data loss is minimized to a backup period timespan.
Fit Criterion: Restoring the latest backup into a clean environment reproduces the project, team, and approval records.
Traces To: NFR_Recoverable

SSR_MultipleBrowseViews The platform shall provide multiple ways to find, filter, and browse the list of projects.
Rationale: While a recommendation engine is proposed, it may be flawed or operate on outdated data. To ensure users don't miss a project that they may favour, providing multiple ways to display the list can bring different projects to the top.
Fit Criterion: The catalog offers search, filters, and at least two sort orders, and every open project can be reached without using recommendations.
Traces To: FR_ViewApprovedProjects

SSR_AtomicActions All system actions which involve multiple dependent reads and writes, with at least one write, shall be atomic.
Rationale: System actions which involve multiple steps may fail at any point, leaving the system in a partially

corrupt state. Atomicity guarantees that an operation either fully completes, or fully fails. Operations which fail can then simply be retried.
Fit Criterion: A fault injected after the first write of slot reservation, slot release, and approval leaves either all or none of that action's changes.
Traces To: NFR_ConcurrencySafety

SSR_NoStaleReads The system shall ensure that operations on sets or aggregations of multiple data entries do not operate on stale data at any point during an operation.
Rationale: Concurrent operations which rely on aggregations of data entries may introduce a data race, resulting in an invalid state after the operation. For example, two students attempting to join the same team which only has one available slot can end up exceeding the team size limit, if the mechanism is not implemented correctly.
Fit Criterion: Over repeated runs, two simultaneous requests to join a team with one free slot produce exactly one success and one explicit failure.
Traces To: NFR_ConcurrencySafety

SSR_NotifyPendingActions The system shall notify the user of pending actions when disabling a component fails.
Rationale: It may be possible to disable certain components but require additional actions to be taken, such as responding to pending project approvals.
Fit Criterion: Attempting to disable the project approval component while at least one approval record is pending is rejected, and the pending records are listed.
Traces To: FR_RejectComponentDisable

SSR_ReverifyCriticalActions The system shall re-verify the user before performing any of the following actions: (1) changing credentials, email, or linked authentication; (2) deleting a proposal, project, profile, thread, or account; (3) changing a user's role or course association; (4) a supervisor or team accepting an agreement; (5) leaving or dissolving a team; (6) a final instructor or TA approval or rejection; (7) confirming an enrollment import or changing course rules or the approval mode. This list is closed; adding an action requires amending this requirement. Re-verification must include at least one of: re-entering credentials, entering an email code, responding to an MFA challenge, and/or responding to an SSO challenge.
Rationale: Sessions may get stolen or unsecured devices may get compromised. Actions which are difficult to undo should be protected to ensure the actual account holder is performing the action.
Traces To: FR_ReverifyUsers
Fit Criterion: For each of the seven actions, attempting it in a session that has not been re-verified prompts for re-verification and does not perform the action until it succeeds.

SSR_EnforceVisibility The system shall check every way of reading a record on the server against the visibility matrix in DPR_VisibilityMatrix. The covered channels are the project catalog, search and filters, project detail pages, threads, notification emails, exported reports, and recommendations.
Rationale: Exposing IP or confidential information can lead to damages and loss of trust. Checking only some channels leaves the others as a way around the restriction.
Traces To: FR_AuthorizationRBAC, DPR_VisibilityMatrix, DPR_RoleBasedDataPermissions
Fit Criterion: For each role and each covered channel, requesting a record that the matrix hides from that role is denied, and no notification email or export contains a field the recipient is not permitted to read.

SSR_ProposalPromptCoverage The proposal template shall require every Project attribute that the SRS data dictionary (FR_SupervisorProposal) marks as Required (background, objectives, impact, completion status, industry, supervisor capacity, confidentiality, and IP) and shall prompt for the Optional attributes.
Rationale: Prompting only an open-ended description gives supervisors no information on what students and instructors are looking for in a capstone-level project, and omits what students need to judge a project (for example how much already exists).
Traces To: FR_SupervisorProposal, FR_ProposalOptionalFields
Fit Criterion: Submission is blocked, with the empty field highlighted, if any Required attribute is empty, and succeeds when only Optional attributes are empty.

SSR_ZeroDefaultPrivileges The system shall assign new users zero privileges when their identity cannot be assigned automatically or verified. The exception is a new sign-in through the external authentication provider, which is assigned the unverified external supervisor role.
Rationale: No assumptions can be made about new users. They may be students who are not enrolled in a

capstone course, malicious actors attempting to extract user information, individuals posing as supervisors to get free labour, etc. Ensuring no unverified users get access to student information mitigates those risks. Unverified external supervisors need only course reference material and their own proposals to submit a proposal.
Traces To: FR_MacSSO, FR_AuthorizationRBAC, FR_UpdateRoles, NFR_Authorize, FR_ExternalSupervisorVerification
Fit Criterion: A McMaster account with no assigned role receives an authorization error on every read and write of course data. A new external sign-in can read only course reference material, the proposal template, and its own proposals.

SSR_ExternalSupervisorVerification The system shall give an external supervisor access to student profiles and student-proposed projects only after an instructor approves a project proposal that the external supervisor submitted, and shall record the verification in the audit log. No other action shall verify an external supervisor.
Rationale: An external identity provider cannot confirm that a person is an industry sponsor, and student information must not be available to anyone who signs in. The instructor's review of a real proposal is the check.
Traces To: FR_ExternalSupervisorVerification, FR_UpdateRoles
Fit Criterion: A new external account requesting a student profile receives an authorization error. It can read one after an instructor approves its proposal, but not if a student proposal naming it is approved, and the verification appears in the audit log.

SSR_ExternalSupervisorScope The system shall restrict an external supervisor, on every channel listed in SSR_EnforceVisibility, to the access specified in the SRS (FR_ExternalSupervisorScope): unverified external supervisors to course reference material, the proposal template, and their own proposals, and verified ones additionally to student profiles, student-proposed projects, and the applications to their own projects. Requests for any other supervisor's proposal or project shall be answered as not found.
Rationale: Industry project ideas are confidential, and any person can obtain an external account.
Traces To: FR_ExternalSupervisorScope, DPR_VisibilityMatrix
Fit Criterion: For each channel, an external supervisor requesting another sponsor's project receives a not-found response, and that project does not appear in any list, search result, count, or email to them. An unverified external supervisor is denied student profiles on every channel.

SSR_PersonalDataDeletion The system shall delete personal data on request and at the end of a course cycle, and shall collect only the data that a feature needs. Audit log entries are retained, with the user reference anonymized.
Rationale: Keeping personal data longer than needed increases the harm of a breach. Anonymizing rather than deleting audit entries keeps the audit log trustworthy.
Traces To: NFR_EssentialPersonal, DPR_UserManagedContentDeletion, DPR_EndOfTermStudentDataDeletion, DPR_ConsentWithdrawal
Fit Criterion: After a deletion request or the end-of-term job, personal fields are absent from the database and from exports (except data covered by consent to retain), and audit entries remain with no identifying user reference.

SSR_RespectfulDeclines The system shall present decline and rejection decisions with respectful message templates and shall not publish rankings of students or teams.
Rationale: Blunt or public decisions cause stress and a sense of unfair treatment, and may stop students from using the platform.
Fit Criterion: Every decline or rejection message is generated from a template, and no page ranks students or teams.
Traces To: FR_DeclineTeam

SSR_EnrollmentPreview The system shall show a preview of any enrollment import and shall apply it only after course staff confirm it.
Rationale: An unchecked import could wrongly add or exclude students and give misleading team validation results.
Fit Criterion: An import changes no enrollment record until the user confirms the preview.
Traces To: FR_ImportEnrollment

SSR_EnrollmentFallback If an enrollment import fails or does not include a student, the system shall leave existing enrollment unchanged, tell the user, and let course staff add students from a file or manually.
Rationale: Avenue may be unavailable, may not exist for a course, or may lack late-joining students, and course staff must still be able to populate the course.

Fit Criterion: With Avenue unavailable, existing enrollment is unchanged, an error is shown, and file upload and manual add both work.
Traces To: FR_LogExternalErrors, FR_ImportEnrollment, FR_UpdateEnrollment

SSR_NoAutoRemoval The system shall not remove a student from a course or team because the student is absent from an enrollment import, and shall flag such students for confirmation by course staff.
Rationale: Automatic removal could leave a team undersized or invalid in the middle of approval.
Fit Criterion: A student on a team who is absent from the latest import stays on the team and appears on a review list.
Traces To: FR_EnrollmentDiscrepency

SSR_EnrollmentValidation The system shall check imported enrollment for duplicate and malformed student entries and show any problems in the preview before the import can be confirmed.
Rationale: Stale or partial data from an external source can corrupt enrollment records.
Fit Criterion: Duplicate and malformed entries are listed in the preview and block confirmation until resolved.
Traces To: FR_ImportEnrollment

SSR_ApprovalStatusIntegrity The system shall display team-project status only from the stored status, using the lifecycle labels in the SRS, and shall hold a change to an approved record as pending until the instructor accepts it.
Rationale: Users act on the status they see. Showing Approved before approval, or altering the approved record early, leads teams and supervisors to act on an approval that does not exist.
Traces To: FR_ViewProjectStatus, FR_ApprovalRevision, FR_RequestApproval
Fit Criterion: A team in each non-Approved status is never displayed as Approved, and a submitted change to an approved record leaves the approved values unchanged until the instructor accepts it.

SSR_AdapterSyncStatus For records whose approval comes from an external approval system, the system shall show the time of the last successful synchronization, flag a failed synchronization, and allow the instructor or TA to record the outcome manually, with the manual change logged.
Rationale: A stale or failed sync makes Keystone disagree with the system that holds the real decision, and users need to see that the data may be out of date.
Traces To: FR_ExternalApproval
Fit Criterion: With the external system made unavailable, the record shows the last-synced time and a failure flag, and a manual override by course staff is saved with an audit entry.

SSR_NotificationRecipientScope The system shall determine each notification's recipients at send time from the user's current role and association with the item, and shall limit the email body to an event summary and a link, excluding confidential project, IP, and profile fields.
Rationale: A misaddressed email can disclose confidential information outside the platform, where visibility rules cannot be enforced.
Traces To: FR_ThreadsNotify, DPR_PreventUnauthorizedDisclosure
Fit Criterion: After a user's role or association with an item is removed, no further notifications for that item are sent to them, and sampled emails contain no confidential fields.

SSR_NotificationDeduplication The system shall give each notification a unique identifier and shall not send the same notification to the same recipient more than once, including when delivery is retried.
Rationale: Durable delivery retries until success, which can duplicate messages if a confirmation is lost.
Traces To: FR_DurableNotifications
Fit Criterion: When delivery confirmation is suppressed and delivery retried, the recipient receives the notification exactly once.

SSR_RuleChangeImpact The system shall validate the course team formation rules on save (minimum size at most maximum size, at least one permitted program combination), restrict editing them to instructors, show the teams that a change would make non-compliant before it is applied, and flag those teams for instructor review rather than invalidating them.
Rationale: Wrong or retroactive rules can block valid teams or silently invalidate teams in the middle of approval.
Traces To: FR_ValidateTeamRules, FR_DisableFeatures
Fit Criterion: A configuration with minimum above maximum is rejected, a change that affects existing teams shows them in a preview, and after applying it those teams remain intact and appear on a review list.

7 Roadmap

The safety and security requirements follow the phases of the development plan in the SRS. A requirement is implemented in the same phase as the feature it protects, so that no feature is released without the protection that its hazards call for. Requirements that protect features outside the course timeline, or that depend on parts of the system that are faked for the demo, are future work.

Within a phase, the order of requirements follows the dependencies between features in the SRS development plan, not a computed risk score. Any requirement deferred to future work is listed below with the reason it can safely wait.

7.1 Phase 1: Core Workflow (implemented in the course)

Phase 1 covers proposal submission and approval, browsing projects with a visible status and team selection, team project submission and review, and the validated team composition export.

- SSR_ProposalPromptCoverage: the proposal template asks for the information that makes a project viable as a capstone project (H_MisleadingDescription).
- SSR_EnforceVisibility: server-side visibility checks for proposals and projects, tested for each role (H_ConfidentialExposure).
- SSR_ZeroDefaultPrivileges: new accounts have no privileges until their role is assigned (H_WrongRole).
- SSR_AuditLog: an append-only audit log for approval decisions, status changes, and role changes (H_WrongRole). Logging every user action is future work.
- SSR_ExternalSupervisorVerification, SSR_ExternalSupervisorScope: the unverified state for every external sign-in, verification through approval of the supervisor's own proposal, and external-supervisor scoping on every read path (H_UnvettedExternalAccess, H_CrossSponsorExposure). These stay in scope even though the demo uses simplified sign-in, because the role checks do not depend on the real identity provider.
- SSR_RecordTimestamps: last-updated time shown on project records (H_OutdatedProject).
- SSR_MultipleBrowseViews: several ways to sort and filter the project list when browsing (H_IrrelevantRanking). Ranked recommendations themselves are future work.
- SSR_AtomicActions, SSR_NoStaleReads: database transactions and locking for team selection, approvals, and team membership (H_StaleProjectStatus, H_TeamSizeRace).
- SSR_ExternalErrorLogging: logging of errors from external providers (H_AuthProviderOutage). Notifying instructors by email is added with notifications in Phase 2.
- SSR_AutomatedBackups: automated database backups by the hosting provider, and a restore test before release (H_DataLoss).
- SSR_RespectfulDeclines: decline and rejection message templates, with no public rankings (H_BluntRejection).
- SSR_EnrollmentPreview, SSR_EnrollmentFallback, SSR_NoAutoRemoval, SSR_EnrollmentValidation: preview and confirmation, fallbacks, flagging, and checks for imported enrollment (H_ImportUnavailable to H_LateStudent). Enrollment import is part of Phase 1 in the SRS.
- SSR_ApprovalStatusIntegrity: only the stored team-project status is displayed, and changes to approved records are held until accepted (H_MisleadingApprovalStatus).
- SSR_RuleChangeImpact: validation of course rules and a preview of affected teams before a rule change is applied (H_RuleMisconfiguration).

7.2 Phase 2: Communication and Visibility (implemented in the course)

Phase 2 adds notifications and the checklist, team profiles and contact details, the course schedule, and identifying unselected projects.

- SSR_DurableNotifications: pending notifications are stored in the database until delivery is confirmed, with retries (H_LostNotification).

- SSR_ExternalErrorLogging: alerts to the instructor when an external provider fails (H_AuthProviderOutage).
- SSR_EnforceVisibility: visibility checks are extended to team profiles, contact details, and unselected projects (H_ConfidentialExposure).
- SSR_NotificationRecipientScope, SSR_NotificationDeduplication: recipients resolved at send time, summary-only email bodies, and unique notification identifiers (H_MisaddressedNotification, H_DuplicateNotification).

7.3 Phase 3: Extended Features (implemented if time allows)

- SSR_ProposalPromptCoverage: the proposal prompts are extended to student-initiated proposals, with the guidelines and examples that this phase adds (H_MisleadingDescription).
- SSR_AtomicActions, SSR_NoStaleReads: the same locking and transactions are applied to supervisor capacity limits (H_StaleProjectStatus, H_TeamSizeRace).

7.4 Future Work (not implemented, or faked for the demo)

- SSR_RejectDisable, SSR_NotifyPendingActions: rules for disabling components apply once features can be switched on and off for each course (modularity and other courses). Until then the hazard (H_DisableBreaksWorkflow) cannot occur.
- SSR_ReverifyCriticalActions: re-verification with email codes or MFA depends on real authentication (McMaster accounts and external accounts), which is simplified for the demo. In the course the system only asks for confirmation before the destructive actions on the list (H_SessionTheft). The deferral is acceptable because demo accounts hold no real student or sponsor data.
- SSR_AdapterSyncStatus: needed only when an adapter to an external approval system is built. Until then approvals are recorded manually, which is the case this requirement's manual override already covers (H_AdapterSyncFailure).
- SSR_MultipleBrowseViews: ranking and explaining recommendations, with matching of teams to projects (H_IrrelevantRanking). The Phase 1 sorting and filtering already keep every project reachable without recommendations.
- SSR_AuditLog: full action logging and tamper-evident storage. Phase 1 logs the actions whose outcomes may need review or reversal (approvals, status changes, role changes); logging every other action adds volume without supporting a decision.
- SSR_ExternalErrorLogging: notifying university IT of provider outages. In the course deployment the instructor is notified in Phase 2; contacting IT depends on a hosting arrangement outside the course.
- SSR_PersonalDataDeletion: deletion on request and at the end of a course cycle (H_ExcessRetention). The demo uses seeded data and holds no real student information, so retention harm arises only once a real deployment stores it. Minimal collection (NFR_EssentialPersonal) applies from Phase 1.
- Hazards of any integration with an external approval system beyond H_AdapterSyncFailure need requirements of their own when it is built.

8 Appendix — Generative AI Use Disclosure

Used Claude Sonnet 5.5 for the following,

- Asked it what are the common security risks apps face that collect profile data to come up with some ideas and reinforce some of my own.
- Used it to get more research into how data breaches can happen as well as data leaks.
- To draft a latex template for the FMEA table.
- Used it to check both documents against the checklist and rubric and asked it to flag anything that was missing and any grammar issues

Verification of AI-Generated Content

The team reviewed every AI-assisted passage before including it. References were checked against the publisher or standards body listing. The FMEA rows, requirements and roadmap were checked by the team against the SRS, including each requirement name in the "Traces To" lines. The team is responsible for the accuracy, completeness, and consistency of this document.

9 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

Waqar:

1. What went well while writing this deliverable? During this deliverable, distributing work was very straightforward as there was a lot of work, but it was separable by section. It was all work we have seen before in prior courses with documentation, so it was very intuitive to write and come up with solutions/answers.
2. What pain points did you experience during this deliverable, and how did you resolve them? During this deliverable, everyone was very busy because we had midterms and a lot of assignments due alongside this report. Sometimes it also became difficult to stay updated with everyone's parts. However, because we planned out our schedule well in advance, we were able to finish everything on time while also completing our other work. Everyone also posted updates in our team chat when they got the chance, so we all stayed in the loop.
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about? Before this deliverable we had thought about things like security issues or our app crashing, which would cause downtime. While working on the hazards, we came up with how a bug could lead to incorrect decisions being made in the actual workflow of our app, which would be chaotic. We also came up with the idea of security regarding the profiles of each person, as we wanted to collect some information to help supervisors get a better idea of who they are working with (the students).
4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider? First, data leakage is always a concern with any software product. It drives whether users can trust the developers. If we have a leak, how we handle it and what measures we take to prevent it show users that we want to protect them and that using the app is safe. Second, availability matters because the system handles time-sensitive information such as deadlines. We want the greatest uptime we can achieve so users do not have to worry about the app going down. Lastly, data correctness is important: while we are handling PII, we want to process it correctly and avoid sharing it with third parties without user consent.

Ayush:

1. What went well while writing this deliverable?

   A great part about this document is it really made us think about our project from a different perspective. It made us think about all the ways things could go wrong, maybe an external service failed, maybe a user did something wrong. It made us think about making our design more robust so it can handle external failures. It also made us think about internal system failures that could have drastic impacts on functionality, such as RBAC not working properly and data integrity concerns. Overall, it was actually quite intuitive to write this deliverable.
2. What pain points did you experience during this deliverable, and how did you resolve them? The main issues with this deliverable were related to keeping everything consistent with the hazard analysis and the SRS. The scope of the system outlined in the SRS dictates the scope of the HA. The hazards and mitigation strategies outlined in the FMEA table must map directly to either functional, non-functional or data requirements in the SRS. But because everything is constantly changing while we are writing it, it was quite difficult to make sure both documents aligned.
3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did

they come about?

Going in, we mostly had the obvious software risks in our heads. We knew that login and permissions (SSO, external accounts, who can see what) could go wrong. We also worried about privacy via student profiles leaking, and supervisors' or students' project ideas getting "stolen". And we knew that if the platform broke, team formation and approvals would get delayed.

Most of the more specific risks only showed up once we went component by component through the FMEA. For example, we only thought about hurt feelings and fairness when we went through the HA slides and checklist. Additionally, we had not thought about data retention policies until we properly analyzed the persistence module.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

   One risk is security and privacy risk. This covers things like someone getting into an account they shouldn't, or personal info and confidential project ideas leaking. It matters because people trust the platform with their data, and once that trust is gone they stop using it. There can also be legal consequences, since software systems have to follow privacy law.

   Psychological and social risk is a big risk in software systems. This is stress, embarrassment, feeling unfairly treated or excluded by how the software behaves. It matters because software shapes how people feel about a process. Something like a blunt rejection message or a visible ranking can really hurt someone, even though nothing "broke".

Ahmed:

1. What went well while writing this deliverable?

   Discovering new requirements which would have been, at best, implicit, really highlighted the value of a hazard analysis. Additionally, it also revealed that some hazards, such as storing invalid states, span multiple components and may require adjusting whole system architecture to completely mitigate.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   It was difficult accounting for all hazards, because of the complexity of the system as represented by its current requirements. The system needs to be modular, secure, maintain data integrity, provide administrator controls, among many others, which, not only have potential hazards within each component, but also raise potential hazards between components. For example, any networked component which does not reject unsecured connections compromises the entire system.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?

   Data security was a clear risk since the platform's most essential data to store and display is sensitive information such as student information, project ideas which may be intellectual property, etc.

   A non-obvious risk which was only raised during the hazard analysis was the data integrity hazard. Two students attempting to join the same team at the same time can end up corrupting the state of the system if not enough care is taken.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

   Data corruption risk is very real and plagues many production applications and can arise due to improper data handling. Data corruption is important to pay attention to because it's often silent and can be unrecoverable if the system does not have any recoverability mechanisms, which require prior planning.

   Another important risk to consider in software products is the user interface aspect. User interfaces which are not intuitive, require manuals or training to understand, or do not account for accessibility requirements, often discourage users from the platform and on a large scale, this can lead to the platform failing completely.

Andrew:

1. What went well while writing this deliverable? What went well while writing this deliverable was the learning experience of thinking about a Hazard Analysis for the first time. It was interesting to take a more critical approach when analyzing our project from a requirements perspective. I learned that it is important to understand what things could go wrong with your project including project discovery failures, misuse and security breaches, incorrect functionality, and loss of data. Identifying these will help us build a security and hazard-free system.

2. What pain points did you experience during this deliverable, and how did you resolve them? The main pain points with this document was that it was very new to us. It is initially difficult to distinguish the content in here with what is already in the SRS. It was also a struggle finding hazards since our system is not safety-critical. At the same time, it was also difficult to tell what information we needed to collect first (including in the SRS) before we analyzed the hazard categories.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about? From the listed risks, we had already thought of security breaches, loss of privacy, and loss of data. We were able to think of these based on our own experiences as full-stack developers. One of the most important things in an application is how its data will be handled. We have all heard of many stories about big companies that went through major issues when it came to their data disclosure and 3rd party access. We knew that securing our clients' data would be of utmost priority since there is sensitive information that users would want to keep securely stored.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider? 1. Loss or corruption of project information: They include failed updates or inconsistent records related to user information, project data, or selected documentation. When stakeholders use our product, they expect that our system retains their records securely. If their project, team, and profile related data is corrupted, the system becomes unusable. 2. Disclosure or misuse: If any data leaks were to occur, personal information and any communications that users may have had through the system would become public. This exposure is undesired and could lead to serious confidentiality and privacy concerns.

References

Louis M. Ippolito and David Wallace. A study on hazard analysis in high integrity software standards and guidelines. Technical Report NISTIR 5589, National Institute of Standards and Technology, Gaithersburg, Maryland, 1995.

Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety.* The MIT Press, Cambridge, Massachusetts, 2011. doi: 10.7551/mitpress/8179.001.0001.